

Chapter 46. Pipelines & Orchestration

Riverstone's daily pipeline in Dagster: dependencies, partitions, a deliberate failure test, backfills, retries, alerts, and delivery only after the data passes its checks.

Draft chapter

17 September 2026

Every output from a real run

Contents

Chapter 46. Pipelines & Orchestration

Why this matters

In plain English

46.1 From scheduled scripts to pipelines

46.2 The core ideas

46.3 Building Riverstone's daily pipeline

46.4 Idempotency, proven with a failure test

46.5 Partitions and backfills

46.6 Retries, timeouts, and alerting

46.7 Delivery only after the data passes its checks

46.8 The same pipeline in Airflow

46.9 Choosing an orchestrator

46.10 Operating pipelines

Common mistakes and how to spot them

In the real world: the Flash that went out twice

Tools

The project: orchestrate Riverstone's ingestion

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 46. Pipelines & Orchestration

Part V — Data Engineering, Integration & Scale

Chapter at a glance

You will learn to: explain what an orchestrator adds to scheduled scripts · describe a pipeline as a graph of assets with dependencies, schedules, and partitions · build Riverstone's daily pipeline in Dagster, from ingestion to a delivered Daily Sales Flash · prove a pipeline step is idempotent with a deliberate failure test, and show what happens when it isn't · rebuild history with a backfill without re-sending old reports · use retries for transient failures without hiding real bugs · stop delivery when the data fails its checks, and send an alert that tells someone what to do · handle late-arriving corrections · read and write the same pipeline in Airflow · choose an orchestrator, and run pipelines responsibly.

Before you start: Chapter 45 (ingestion, hashes, upserts, idempotency), Chapter 29 (Python as software), and Chapter 20 (automating reports).

Time needed: 14–18 hours of reading and practice, spread over two to three weeks.

Tools: Python 3.12 with `dagster`, `duckdb`, `psycopg2`, and `requests`; PostgreSQL 16; and the Chapter 46 companion folder. Everything runs on your own computer.

Practice data: the `riverstone_source` database from Chapter 45 (a private copy of the one-year database, playing the ERP and CRM), Bhiwandi Main's dispatch files, and the practice CRM API. Every output shown is from a real run.

Why this matters

In Chapter 20, the Daily Sales Flash became a script that runs every morning and emails the managers. In Chapter 45, you built the loads that bring orders, dispatches, and leads into the warehouse. Each piece works on its own. Now they have to work *together*, every day, in the right order, without anyone watching.

That's harder than it sounds. The dispatch file arrives late. The CRM drops a connection. The order-line load hits a bug on the same morning a manager is waiting for the numbers. A script scheduled for 6:30 a.m. doesn't know that the orders it's reporting on haven't finished loading. And when something fails at 6:31, someone has to know what failed, what's safe to re-run, and whether the managers already received a wrong email.

An **orchestrator** is the software that runs these steps in the right order, on schedule, and keeps a record of every run. Good orchestration turns a collection of scripts into a **pipeline**: something you can trust, re-run, repair, and explain. This chapter builds Riverstone's first real pipeline, breaks it on purpose in four different ways, and shows how a well-designed pipeline survives each one.

In plain English

Think about how a restaurant kitchen gets dinner out.

- The **recipe** says what depends on what: the sauce needs the stock, the plate needs the sauce and the rice. That's the **dependency graph** of a pipeline.
- The **head chef** doesn't cook everything. The head chef makes sure each station starts at the right time, in the right order, and knows when something's gone wrong. That's the **orchestrator**.
- Each **service** (Monday lunch, Monday dinner) is prepared separately. That's a **partition**: one day's run of the pipeline.
- If the rice burns, you remake the rice. You don't serve two plates of rice, and you don't throw away the sauce. Remaking a step without doubling anything is **idempotency**.
- Before a plate leaves the kitchen, someone checks it. A bad plate goes back; it never reaches the table. That's a **check that blocks delivery**.
- If the fish supplier is late, a runner calls again in five minutes. That's a **retry**. But if the fish is off, calling again won't fix it.
- Reprinting last month's menus for the archive isn't the same as handing them to today's customers. Rebuilding history without re-sending it is a **backfill**.

The Daily Sales Flash is a plate that leaves the kitchen every morning at 6:30. This chapter is about making sure it's the right plate.

46.1 From scheduled scripts to pipelines

The simplest scheduler is **cron**, the tool on Linux and macOS that runs a command at set times. A cron line like `30 6 * * * python3 daily_flash.py` runs a script at 6:30 every morning. Windows Task Scheduler and Chapter 20's scheduled Apps Script triggers do the same job.

For a single script, that's often enough. Problems start when there are several steps that depend on each other:

- **Order.** If orders load at 6:00 and the Flash runs at 6:30, what happens on the day orders take 40 minutes?
- **Failure handling.** If the order-line load fails, does the Flash still run on stale data?
- **Retries.** If the CRM drops the connection, who tries again?
- **Re-running.** If you fix a bug and re-run, do you get duplicate rows or duplicate emails?
- **History.** Which runs succeeded last week? How long did each step take? What did it load?
- **Backfills.** How do you rebuild March after fixing a calculation, without re-sending March's emails?
- **Visibility.** Which reports depend on the dispatch file? If it breaks, what else is affected?

An orchestrator answers all of these in one place.

Need	Cron plus scripts	An orchestrator
Run steps in dependency order	You wire it by hand, often with time gaps	Declared once; the orchestrator waits for upstream steps
Skip downstream steps when something fails	Each script has to check	Built in
Retry transient failures	Custom code in every script	A retry policy per step
Run history, logs, durations	Scattered log files, if any	Every run recorded and searchable
Re-run one day, or rebuild a date range	Manual, error-prone	Partitions and backfills
See what depends on what	In people's heads	A graph you can view
Alert the right person	Custom code	Failure hooks and sensors

Simplification note. Cron isn't wrong. A single report with no dependencies, run by one person who checks it, doesn't need an orchestrator. Section 46.9 covers when plain scheduling is the better choice.

46.2 The core ideas

Pipelines as graphs

A pipeline is a **directed acyclic graph** (DAG): a set of steps connected by arrows that show which step needs which, with no loops. "Acyclic" means you can never follow the arrows back to where you started, so there's always a valid order to run things in.

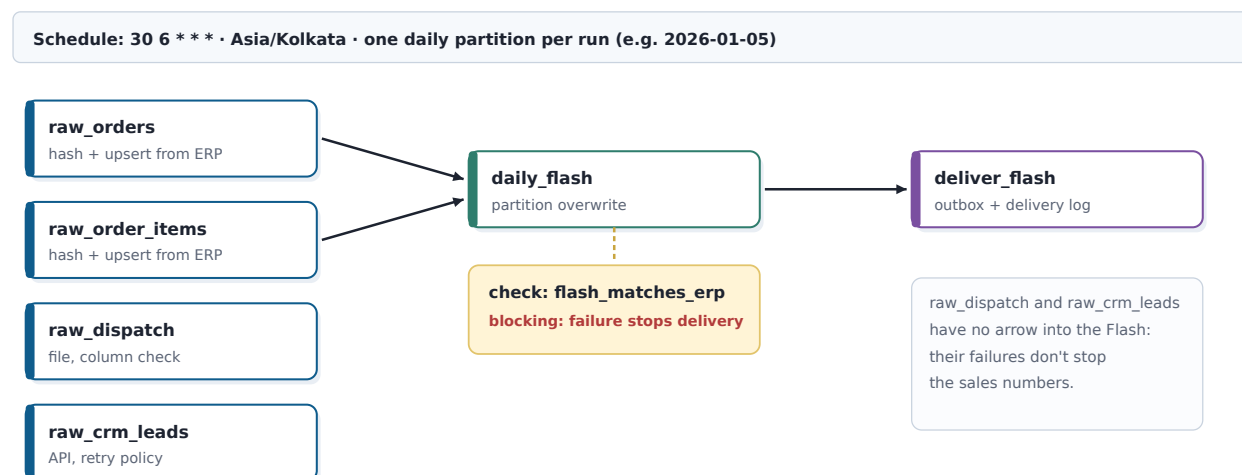


Figure 46.1 – Riverstone's daily pipeline as a graph. Notice what the Flash does and doesn't depend on: a problem with the dispatch file shouldn't stop the sales numbers.

Tasks and assets

There are two ways to think about the steps.

- **Task-based** orchestrators, such as Airflow, describe *what to run*: "run the load script, then run the report script". The data each task produces is implied.
- **Asset-based** orchestrators, such as Dagster, describe *what should exist*: "the table `raw.orders` should exist, built from the ERP; the `daily_flash` table depends on `raw.orders` and `raw.order_items`". An **asset** is a piece of data that the pipeline produces and keeps: a table, a file, a model, a report.

Both work, and both end up as a DAG. The asset view makes it easier to answer questions such as "which reports use this table?" and "is this table up to date?", which is why this chapter builds in Dagster. Section 46.8 shows the same pipeline written as Airflow tasks.

Schedules, time zones, and partitions

A **schedule** says when a pipeline runs. It's usually written as a **cron expression**: five fields for minute, hour, day of month, month, and day of week.

Expression	Meaning
<code>30 6 * * *</code>	6:30 every day
<code>0 7 * * 1</code>	7:00 every Monday
<code>0 */2 * * *</code>	Every two hours, on the hour
<code>15 22 1 * *</code>	10:15 p.m. on the 1st of every month

A cron expression means nothing without a **time zone**. Servers usually run in **UTC** (Coordinated Universal Time); Riverstone runs on **IST** (India Standard Time, `Asia/Kolkata`), which is UTC plus 5 hours 30 minutes. A schedule of `30 6 * * *` in UTC would run at noon in Mumbai.

A **partition** is one slice of the data that the pipeline processes as a unit, most often one day. Partitioning means each run knows exactly which day it's building, so you can re-run 5 January without touching 2 January, and rebuild a whole month one day at a time.

Watch out: "yesterday" depends on where you are. At 3:00 a.m. IST on 6 January, it's still 5 January in UTC. A pipeline that computes "yesterday" from the server clock in UTC will build the wrong day for five and a half hours every night. Set the time zone explicitly on schedules and partitions, and pass the partition date into every step instead of reading the clock.

46.3 Building Riverstone's daily pipeline

Setting up

The companion folder `companion/ch46/` reuses Chapter 45's source database and practice API, and adds two things:

File	What it does
<code>reset_ch46.py</code>	Recreates <code>riverstone_source</code> , and empties the warehouse, the <code>outbox</code> folder (where delivered reports go), and the <code>alerts</code> folder
<code>ingest.py</code>	The Chapter 45 loads packaged as functions: <code>sync_table()</code> (hash comparison and upsert), <code>load_dispatch()</code> (with a column check), <code>fetch_leads()</code> (paginated API)
<code>apply_day.py</code> , <code>make_files.py</code> , <code>mock_crm_api.py</code>	Copied from Chapter 45

Install Dagster with `pip install dagster`, start Python in the `companion/ch46` folder, and run:

```
import os, hashlib
import duckdb, dagster as dg
from reset_ch46 import reset
from apply_day import apply_day
from mock_crm_api import start_server
import ingest

STEP_ORDER = ["raw_orders", "raw_order_items", "raw_dispatch", "raw_crm_leads",
              "daily_flash", "flash_matches_erp", "deliver_flash"]
RUN_LOG = []

def log(step, message):
    RUN_LOG.append((step, message))

def show_log():
    """Print this run's messages in pipeline order, then clear them."""
    for step, message in sorted(RUN_LOG, key=lambda e: STEP_ORDER.index(e[0])):
        print(message)
    RUN_LOG.clear()

reset()
wh = duckdb.connect("warehouse/riverstone_wh.duckdb")
api = start_server()
print("environment ready")
```

```
environment ready
```

How it works. Most of this block is the Chapter 45 setup. The new part is the small **run log**: each step records a message with `log()`, and `show_log()` prints them in pipeline order. Steps that don't depend on each other can run in any order, so printing messages as they happen would show them in a different sequence each time. Dagster keeps its own detailed event log too; this one keeps the book's outputs short and readable.

Tool note. Outputs in this chapter were produced with Python 3.12.3, Dagster 1.13.23, DuckDB 1.5.5, psycopg2 2.9.13, requests 2.33.1, and PostgreSQL 16. Dagster's API changes between versions; if a name doesn't exist in your version, check its documentation for the current equivalent.

The ingestion assets

```
daily = dg.DailyPartitionsDefinition(start_date="2026-01-01", end_date="2026-01-06",
timezone="Asia/Kolkata")

@dg.asset(partitions_def=daily, group_name="ingest")
def raw_orders(context):
    new, changed, deleted = ingest.sync_table(wh, "orders")
    log("raw_orders", f"[{context.partition_key}] raw_orders: {new} new, {changed} changed,
{deleted} deleted")

@dg.asset(partitions_def=daily, group_name="ingest")
def raw_order_items(context):
    new, changed, deleted = ingest.sync_table(wh, "order_items")
    log("raw_order_items", f"[{context.partition_key}] raw_order_items: {new} new, {changed}
changed, {deleted} deleted")

@dg.asset(partitions_def=daily, group_name="ingest")
def raw_dispatch(context):
    try:
        outcome = ingest.load_dispatch(wh, context.partition_key)
    except ValueError as e:
        log("raw_dispatch", f"[{context.partition_key}] raw_dispatch: FAILED, {e}")
        raise
    log("raw_dispatch", f"[{context.partition_key}] raw_dispatch: {outcome}")

CRM_FLAKY = {"fail_next": False}

@dg.asset(partitions_def=daily, group_name="ingest",
          retry_policy=dg.RetryPolicy(max_retries=3, delay=1))
def raw_crm_leads(context):
    if CRM_FLAKY["fail_next"]:
        CRM_FLAKY["fail_next"] = False
        log("raw_crm_leads", f"[{context.partition_key}] raw_crm_leads: connection dropped (attempt
{context.retry_number + 1})")
        raise ConnectionError("CRM connection reset")
    leads = ingest.fetch_leads()
    wh.execute("CREATE SCHEMA IF NOT EXISTS raw")
    wh.execute("CREATE TABLE IF NOT EXISTS raw.crm_leads (lead_id INTEGER PRIMARY KEY, company_name
VARCHAR, source VARCHAR)")
    wh.executemany("INSERT OR REPLACE INTO raw.crm_leads VALUES (?, ?, ?)",
                  [(l["lead_id"], l["company_name"], l["source"]) for l in leads])
    log("raw_crm_leads", f"[{context.partition_key}] raw_crm_leads: {len(leads)} leads (attempt
{context.retry_number + 1})")
```

How it works, line by line.

- `DailyPartitionsDefinition` declares that this pipeline runs one day at a time, from 1 January to 5 January 2026 (the end date isn't included), **in IST**. A real pipeline would have no end date.
- `@dg.asset` turns a function into an asset. The function's name becomes the asset's name. `partitions_def=daily` makes it daily; `group_name` groups related assets in Dagster's web interface.
- `context` is passed in by Dagster. `context.partition_key` is the day being built, as text: "2026-01-02". Every step uses it instead of reading the clock.
- The four assets call the Chapter 45 functions. Each is idempotent: running it again with no source changes changes nothing.

- `raw_crm_leads` has a **retry policy**: if it raises an error, Dagster tries again up to three more times, one second apart. `context.retry_number` counts retries from 0. The `CRM_FLAKY` switch lets section 46.6 simulate a dropped connection.

The Flash, its check, and delivery

```
FLASH_SQL = """
    SELECT COUNT(DISTINCT o.order_id)                                AS
orders_booked,
        COALESCE(SUM(i.quantity * i.unit_price * (1 - i.discount_pct / 100)), 0) AS
revenue_booked
    FROM {orders} AS o
    JOIN {items} AS i ON i.order_id = o.order_id
    WHERE o.order_date = {day}
        AND o.status <> 'Cancelled'"""

@dg.asset(partitions_def=daily, group_name="report", deps=[raw_orders, raw_order_items])
def daily_flash(context):
    day = context.partition_key
    wh.execute("CREATE SCHEMA IF NOT EXISTS mart")
    wh.execute("""CREATE TABLE IF NOT EXISTS mart.daily_flash (
        flash_date DATE, orders_booked INTEGER, revenue_booked DECIMAL(12,2))""")
    wh.execute("BEGIN")
    try:
        wh.execute("DELETE FROM mart.daily_flash WHERE flash_date = ?", [day])
        wh.execute("INSERT INTO mart.daily_flash SELECT DATE '" + day + "', * FROM (" +
            FLASH_SQL.format(orders="raw.orders", items="raw.order_items", day="?") + ")",
[day])
        wh.execute("COMMIT")
    except Exception:
        wh.execute("ROLLBACK")
        raise
    n, rev = wh.execute("SELECT orders_booked, revenue_booked FROM mart.daily_flash WHERE
flash_date = ?", [day]).fetchone()
    log("daily_flash", f"[{day}] daily_flash: orders={n}, revenue=Rs {rev:,.2f}")

@dg.asset_check(asset=daily_flash, blocking=True)
def flash_matches_erp(context):
    day = context.partition_key
    src_n, src_rev = ingest.fetch_rows(
        FLASH_SQL.format(orders="orders", items="order_items", day="%s"), [day])[0]
    rows = wh.execute("SELECT orders_booked, revenue_booked FROM mart.daily_flash WHERE flash_date
= ?", [day]).fetchall()
    passed = len(rows) == 1 and rows[0][0] == src_n and rows[0][1] == src_rev
    wh_text = f"orders={rows[0][0]}, revenue=Rs {rows[0][1]:,.2f}" if len(rows) == 1 else f"
{len(rows)} rows"
    log("flash_matches_erp", f"[{day}] check flash_matches_erp: {'PASSED' if passed else 'FAILED'}
"
        f"(ERP: orders={src_n}, revenue=Rs {src_rev:,.2f} | warehouse:
{wh_text})")
    return dg.AssetCheckResult(passed=passed)

@dg.asset(partitions_def=daily, group_name="report", deps=[daily_flash])
def deliver_flash(context):
    day = context.partition_key
    n, rev = wh.execute("SELECT orders_booked, revenue_booked FROM mart.daily_flash WHERE
flash_date = ?", [day]).fetchone()
    body = f"Riverstone Daily Sales Flash - {day}\nOrders booked: {n}\nRevenue booked (net of
discounts, excl. cancelled): Rs {rev:,.2f}\n"
    digest = hashlib.sha256(body.encode()).hexdigest()
    wh.execute("CREATE TABLE IF NOT EXISTS mart.delivery_log (flash_date DATE, content_sha VARCHAR,
file_name VARCHAR)")
    sent = wh.execute("SELECT content_sha FROM mart.delivery_log WHERE flash_date = ?",
[day]).fetchall()
    if any(s[0] == digest for s in sent):
        log("deliver_flash", f"[{day}] deliver_flash: already delivered, skipped"); return
    name = f"flash_{day}.txt" if not sent else f"flash_{day}_correction_{len(sent)}.txt"
    with open(os.path.join("outbox", name), "w") as f:
        f.write(body)
```

```
wh.execute("INSERT INTO mart.delivery_log VALUES (?, ?, ?)", [day, digest, name])
log("deliver_flash", f"[{day}] deliver_flash: wrote outbox/{name}")
```

How it works, line by line.

- `FLASH_SQL` is one query with placeholders for table names, so the *same* definition runs against the warehouse and against the ERP. Revenue is $\text{quantity} \times \text{unit price} \times (1 - \text{discount}\%)$, excluding cancelled orders: Riverstone's "bookings" definition from Chapter 3.
- `daily_flash` declares `deps=[raw_orders, raw_order_items]`: it only runs after both have succeeded for the same day. It builds the day's row with **delete-then-insert inside one transaction**, so re-running a day replaces that day's row, and a failure halfway leaves the table as it was. Section 46.4 proves both.
- `@dg.asset_check(asset=daily_flash, blocking=True)` defines a **check** on the Flash. It recomputes the same numbers from the ERP and passes only if the warehouse has exactly one row that matches. `blocking=True` means that if the check fails, **nothing downstream of daily_flash runs**.
- `deliver_flash` depends on `daily_flash`. It writes the report to `outbox/` (standing in for an email, which Chapter 20 showed how to send) and records a hash of the content in `mart.delivery_log`. If exactly that content was already delivered, it skips; if the numbers changed, it sends a clearly labeled correction.

Declaring the job and its schedule

```
ALL = [raw_orders, raw_order_items, raw_dispatch, raw_crm_leads, daily_flash, flash_matches_erp,
deliver_flash]
flash_job = dg.define_asset_job("daily_flash_job", selection=dg.AssetSelection.assets(*ALL[:5],
deliver_flash), partitions_def=daily)
flash_schedule = dg.build_schedule_from_partitioned_job(flash_job, hour_of_day=6,
minute_of_hour=30)
defs = dg.Definitions(assets=[a for a in ALL if a is not flash_matches_erp], asset_checks=
[flash_matches_erp],
                      jobs=[flash_job], schedules=[flash_schedule])
print("job      :", flash_job.name)
print("schedule:", flash_schedule.cron_schedule, flash_schedule.execution_timezone)
print("assets  :", sorted(k.to_user_string() for k in
defs.resolve_asset_graph().get_all_asset_keys()))
```

```
job      : daily_flash_job
schedule: 30 6 * * * Asia/Kolkata
assets  : ['daily_flash', 'deliver_flash', 'raw_crm_leads', 'raw_dispatch', 'raw_order_items',
'raw_orders']
```

How it works. A **job** is a selection of assets to run together; `define_asset_job` builds one from the assets. `build_schedule_from_partitioned_job` creates a schedule that runs the job every day at 6:30 for the day's partition, and takes the time zone from the partitions definition: `30 6 * * * in Asia/Kolkata`. `Definitions` bundles everything, and Dagster validates the graph when it's built, so a missing dependency or a duplicate name fails here instead of at 6:30 a.m.

To see the pipeline in Dagster's web interface, save the definitions in a file and run `dagster dev -f yourfile.py`; it opens a local page showing the graph, the schedule, and every run.

The first run

It's 2 January. The ERP has had its business day. Run the whole pipeline for that day's partition:

```
apply_day(1)
result = dg.materialize(ALL, partition_key="2026-01-02", raise_on_error=False)
show_log()
print("run succeeded:", result.success)
print()
print(open("outbox/flash_2026-01-02.txt").read())
```

```
[2026-01-02] raw_orders: 177 new, 0 changed, 0 deleted
[2026-01-02] raw_order_items: 333 new, 0 changed, 0 deleted
[2026-01-02] raw_dispatch: 5 rows
[2026-01-02] raw_crm_leads: 43 leads (attempt 1)
[2026-01-02] daily_flash: orders=2, revenue=Rs 38,710.00
[2026-01-02] check_flash_matches_erp: PASSED (ERP: orders=2, revenue=Rs 38,710.00 | warehouse:
orders=2, revenue=Rs 38,710.00)
[2026-01-02] deliver_flash: wrote outbox/flash_2026-01-02.txt
run succeeded: True
```

```
Riverstone Daily Sales Flash - 2026-01-02
Orders booked: 2
Revenue booked (net of discounts, excl. cancelled): Rs 38,710.00
```

Reading it. The steps ran in dependency order. The first run loaded all 177 orders and 333 order lines into the empty warehouse, the dispatch file's 5 rows, and all 43 leads. The Flash shows 2 orders booked for ₹38,710.00. The check recomputed the same numbers from the ERP and they matched, so delivery ran and wrote the report.

Dialect note: rupees in outputs. The code prints `Rs` rather than the ₹ symbol, because some terminals and email systems don't display ₹ correctly. The prose uses ₹ as elsewhere in the book.

Running it again

What happens if the scheduler runs the same day twice, or someone presses "run" again?

```
result = dg.materialize(ALL, partition_key="2026-01-02", raise_on_error=False)
show_log()
print("run succeeded:", result.success)
print("flash rows for the day:", wh.execute("SELECT COUNT(*) FROM mart.daily_flash WHERE flash_date
= '2026-01-02'").fetchone()[0])
print("files in outbox:", sorted(os.listdir("outbox")))
```

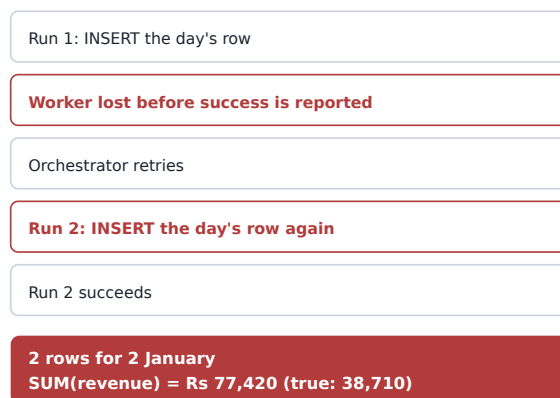
```
[2026-01-02] raw_orders: 0 new, 0 changed, 0 deleted
[2026-01-02] raw_order_items: 0 new, 0 changed, 0 deleted
[2026-01-02] raw_dispatch: 5 rows
[2026-01-02] raw_crm_leads: 43 leads (attempt 1)
[2026-01-02] daily_flash: orders=2, revenue=Rs 38,710.00
[2026-01-02] check_flash_matches_erp: PASSED (ERP: orders=2, revenue=Rs 38,710.00 | warehouse:
orders=2, revenue=Rs 38,710.00)
[2026-01-02] deliver_flash: already delivered, skipped
run succeeded: True
flash rows for the day: 1
files in outbox: ['flash_2026-01-02.txt']
```

Nothing changed: no rows loaded, one Flash row for the day, and delivery recognized that exactly this report was already sent. That's the behavior you want from every scheduled pipeline, and it isn't an accident. The next section shows what it takes.

46.4 Idempotency, proven with a failure test

Chapter 45 defined an **idempotent** step as one you can run again with the same result. In a pipeline, the question that matters is sharper: **if a step fails halfway and is retried, is the result still correct?** The only honest way to answer is to make it fail on purpose.

Naive step: append



Safe step: overwrite in a transaction

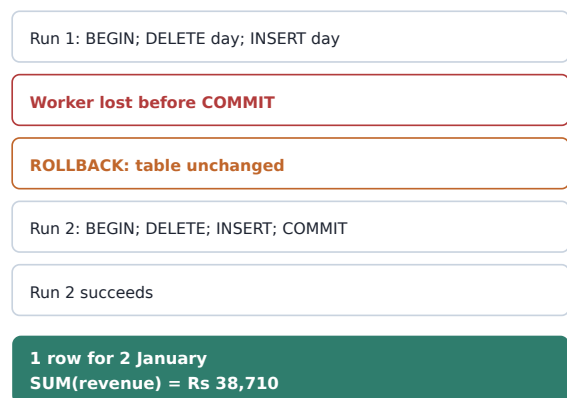


Figure 46.2 — The same failure, two designs. The naive step's retry doubles the day; the safe step's retry is harmless.

A step that isn't idempotent

Here's a common first version of a report step: it **appends** the day's numbers. To test it, the step crashes *after* writing, as happens when a machine is lost or a network call times out after the database has already committed.

```

CRASH = {"now": False}

@dg.asset(partitions_def=daily, deps=[raw_orders, raw_order_items])
def flash_naive(context):
    day = context.partition_key
    wh.execute("CREATE TABLE IF NOT EXISTS mart.flash_naive (flash_date DATE, orders_booked
INTEGER, revenue_booked DECIMAL(12,2))")
    wh.execute("INSERT INTO mart.flash_naive SELECT DATE '" + day + "', * FROM (" +
FLASH_SQL.format(orders="raw.orders", items="raw.order_items", day="?") + ")",
[day])
    if CRASH["now"]:
        raise RuntimeError("worker lost after writing (simulated)")

CRASH["now"] = True
first = dg.materialize([raw_orders, raw_order_items, flash_naive], partition_key="2026-01-02",
raise_on_error=False)
CRASH["now"] = False
second = dg.materialize([raw_orders, raw_order_items, flash_naive], partition_key="2026-01-02",
raise_on_error=False)
RUN_LOG.clear()
print("first run succeeded :", first.success)
print("second run succeeded:", second.success)
print(wh.execute("""SELECT COUNT(*) AS rows_for_day, SUM(revenue_booked) AS revenue_reported
FROM mart.flash_naive WHERE flash_date = '2026-01-02'""").fetchall())

```

```

first run succeeded : False
second run succeeded: True
[(2, Decimal('77420.00'))]

```

Reading it. The first run failed, so the scheduler's retry (here, a second run) did its job and succeeded. But the table now has **two rows** for 2 January, and anything that sums revenue reports ₹77,420.00, exactly double the true ₹38,710.00. Both runs logged honest messages. Nothing looks broken until someone compares the report with the ERP.

The same step, made safe

Now the design from section 46.3: delete the day's rows and insert them again, inside one transaction. The same crash, at the same point:

```

def build_flash_safely(day, crash=False):
    wh.execute("BEGIN")
    try:
        wh.execute("DELETE FROM mart.daily_flash WHERE flash_date = ?", [day])
        wh.execute("INSERT INTO mart.daily_flash SELECT DATE '" + day + "', * FROM (" +
FLASH_SQL.format(orders="raw.orders", items="raw.order_items", day="?") + ")",
[day])
        if crash:
            raise RuntimeError("worker lost after writing (simulated)")
        wh.execute("COMMIT")
    except Exception as e:
        wh.execute("ROLLBACK")
        print("run failed and rolled back:", e)

build_flash_safely("2026-01-02", crash=True)
build_flash_safely("2026-01-02")
print(wh.execute("""SELECT COUNT(*) AS rows_for_day, SUM(revenue_booked) AS revenue_reported
FROM mart.daily_flash WHERE flash_date = '2026-01-02'""").fetchall())

```

```
run failed and rolled back: worker lost after writing (simulated)
[(1, Decimal('38710.00'))]
```

Reading it. When the step crashed, the transaction **rolled back**: the delete and the insert were both undone, so the table was left exactly as it was. The retry then committed. There's one row, with the correct ₹38,710.00.

How it works. A **transaction** groups statements so they succeed or fail together (Chapter 12). `BEGIN` starts one; `COMMIT` makes it permanent; `ROLLBACK` discards everything since `BEGIN`. The combination of **replace the whole partition** and **one transaction** is the standard pattern for idempotent pipeline steps. Chapter 49 covers how warehouses guarantee it, under the name ACID.

Patterns for idempotent steps

Pattern	How it works	Use it for
Partition overwrite	Delete (or overwrite) the partition, then insert, in one transaction	Daily reports, aggregates, anything built per day
Upsert by key	Insert new keys, update existing ones (Chapter 45)	Raw tables copied from sources
Write to a new location, then swap	Build the result as a new table or file; replace the old one in a single step when complete	Large rebuilds, where a half-built table must never be visible
Delivery log	Record what was sent (with a content hash); skip exact repeats	Emails, messages, API calls, CRM updates
Idempotency keys	Send a unique key with each external request so the receiver ignores duplicates	Payments and external APIs that support it (Chapter 51)

Watch out: side effects outside the database can't be rolled back. A transaction can undo a table write. It can't un-send an email, un-post a message, or un-call an API. That's why `deliver_flash` checks a delivery log first, and why delivery should always be the *last* step, after every check has passed.

Try it. Change `CRASH["now"] = True` to crash *before* the insert in `flash_naive` (move the `if` above the `INSERT`), reset, and run the test again. Does the naive step still double the revenue? What does that tell you about when a non-idempotent step is dangerous?

46.5 Partitions and backfills

A **backfill** runs a pipeline for past partitions: to build history for a new pipeline, to rebuild after fixing a bug, or to fill a gap after an outage.

Backfills bring one important rule: **rebuild history, don't re-deliver it**. Nobody wants five old Sales Flash emails arriving at once. So the backfill below runs every asset *except* delivery.

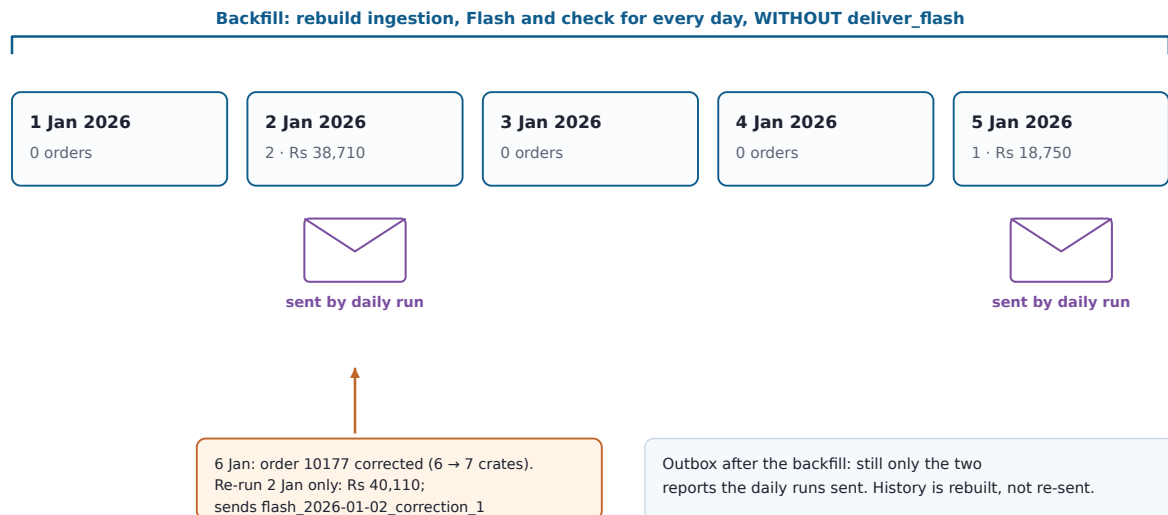


Figure 46.3 – Partitions, a backfill, and a late correction. The backfill fills in history quietly; only real changes to already-delivered numbers produce a new message.

```
BACKFILL = [a for a in ALL if a is not deliver_flash]      # rebuild history, don't email it

for day in daily.get_partition_keys():
    dg.materialize(BACKFILL, partition_key=day, raise_on_error=False)
RUN_LOG.clear()
print(wh.execute("SELECT flash_date, orders_booked, revenue_booked FROM mart.daily_flash ORDER BY
flash_date").fetchall())
print("files in outbox:", sorted(os.listdir("outbox")))
```

```
[(datetime.date(2026, 1, 1), 0, Decimal('0.00')), (datetime.date(2026, 1, 2), 2,
Decimal('38710.00')), (datetime.date(2026, 1, 3), 0, Decimal('0.00')), (datetime.date(2026, 1, 4),
0, Decimal('0.00')), (datetime.date(2026, 1, 5), 0, Decimal('0.00'))]
files in outbox: ['flash_2026-01-02.txt']
```

Reading it. The backfill built a Flash row for every day from 1 to 5 January. Three days have no bookings in the practice data: 1 January, and the weekend of 3–4 January. No new files appeared in the outbox: the only two reports are the ones the normal daily runs delivered.

How it works. `daily.get_partition_keys()` lists every day in the partitions definition. Here the loop runs one day at a time, in order, in this Python session. In a deployed Dagster instance, you'd select a date range in the web interface or run `dagster job backfill` from the command line, and Dagster would queue the runs and track each one.

Backfill etiquette

- **Check that every step is idempotent first.** A backfill runs every step again for every day. A non-idempotent step turns a one-day bug into a month of doubled numbers.
- **Exclude side effects:** emails, CRM updates, messages, API writes. Run them only for the days that truly need re-sending, with a clear "correction" label.
- **Mind the source.** Thirty days of backfill can mean thirty full reads of the ERP. Run large backfills outside business hours, limit how many partitions run at once, and tell the source system's owner.

- **Keep days independent.** A step that reads "the latest" data instead of "the partition's" data gives every backfilled day today's numbers.
- **Record why.** Note who ran the backfill, which dates, and why, next to the run.

Late-arriving data

Data doesn't always arrive on the day it describes. On 6 January, the sales team discovers that order 10177, booked on 2 January, was for 7 industrial crates, not 6, and corrects it in the ERP. The 2 January Flash is now wrong.

Because every step is partitioned and idempotent, the fix is to re-run that one day:

```
conn = ingest.psycpg2.connect(ingest.SRC)
with conn, conn.cursor() as cur:
    cur.execute("UPDATE order_items SET quantity = 7 WHERE order_item_id = 333")
conn.close()

result = dg.materialize(ALL, partition_key="2026-01-02", raise_on_error=False)
show_log()
print(open("outbox/flash_2026-01-02_correction_1.txt").read())
api.shutdown()
```

```
[2026-01-02] raw_orders: 0 new, 0 changed, 0 deleted
[2026-01-02] raw_order_items: 0 new, 1 changed, 0 deleted
[2026-01-02] raw_dispatch: 5 rows
[2026-01-02] raw_crm_leads: 43 leads (attempt 1)
[2026-01-02] daily_flash: orders=2, revenue=Rs 40,110.00
[2026-01-02] check_flash_matches_erp: PASSED (ERP: orders=2, revenue=Rs 40,110.00 | warehouse:
orders=2, revenue=Rs 40,110.00)
[2026-01-02] deliver_flash: wrote outbox/flash_2026-01-02_correction_1.txt
Riverstone Daily Sales Flash - 2026-01-02
Orders booked: 2
Revenue booked (net of discounts, excl. cancelled): Rs 40,110.00
```

Reading it. The order-line load found one changed row. The Flash for 2 January was rebuilt at ₹40,110.00, ₹1,400.00 more than before: one more crate at ₹1,400. The check passed against the ERP. Delivery saw that the content differed from what was sent on 2 January, so it wrote a separate, clearly named correction instead of silently replacing the original.

Deciding how far back to look. A pipeline can't re-run every past day every morning. Common approaches: re-run a fixed **look-back window** each day (for example, the last 7 days), which catches most late changes automatically; re-run specific days when a change is detected (for example, with the hash comparison from Chapter 45 grouped by order date); or accept that some numbers are final after a closing date, as finance teams do with month-end.

46.6 Retries, timeouts, and alerting

Retries: for problems that go away

Some failures are **transient**: a dropped connection, a busy API, a database restarting. Trying again a little later usually works. That's what the retry policy on `raw_crm_leads` is for. Simulate a dropped connection:

```
CRM_FLAKY["fail_next"] = True
result = dg.materialize([raw_crm_leads], partition_key="2026-01-02", raise_on_error=False)
show_log()
print("run succeeded:", result.success)
```

```
[2026-01-02] raw_crm_leads: connection dropped (attempt 1)
run succeeded: False
```

Reading it. The first attempt failed with a dropped connection. Dagster waited one second, retried, and the second attempt loaded all 43 leads. The run succeeded without anyone being woken up.

Retries have a cost, and a trap.

- **Retries hide bugs.** While building this chapter's pipeline, `raw_crm_leads` first failed because it tried to create a table in a schema that didn't exist yet on a brand-new warehouse. The retry succeeded, because another step had created the schema in the meantime. The run looked healthy; the bug was real. Log every retry, and treat frequent retries as a problem to investigate.
- **Retry only what might succeed.** A wrong password, a missing column, or bad data fails the same way every time. Chapter 45's rule applies to whole steps too: retry timeouts and service errors; fail fast on everything else.
- **Retries need idempotent steps.** A retry is a re-run. Section 46.4 is what makes retries safe.

Timeouts: for problems that never end

A step that hangs (waiting forever on a network call, or a query that locks) is worse than one that fails, because nothing downstream runs and no alert fires. Set timeouts at every level: on network calls (Chapter 45), on database statements, and on whole runs, so a run that takes far longer than usual is stopped and reported. Dagster supports run-level timeouts through tags such as `dagster/max_runtime` in a deployed instance; check your version's documentation.

Alerting: telling someone what to do

An **alert** is a message sent to a person when something needs attention. A good alert is:

- **Actionable:** it says what failed, what the impact is, and what to do next.
- **Rare:** alerts that fire for things nobody needs to act on teach people to ignore alerts.
- **Routed:** it goes to the person or team that owns the fix, not to everyone.
- **Linked:** it points to the run, the logs, and a **runbook**, a short document that explains how to diagnose and fix a known problem.

Section 46.7 shows an alert from a real failed run. In a deployed Dagster instance, alerts are usually sent by a **run failure sensor**, which the Dagster daemon checks continuously:

```
@dg.run_failure_sensor(monitored_jobs=[flash_job])
def flash_failure_alert(context: dg.RunFailureSensorContext):
    run = context.dagster_run
    send_message(
        to="data-oncall@riverstone.example",          # your email, Slack, or Teams function
        subject=f"Daily Sales Flash failed: run {run.run_id[:8]}",
        body=f"Failed job: {run.job_name}\nError: {context.failure_event.message}\n"
            f"Runbook: docs/runbooks/daily_flash.md",
    )
```

This block needs a running Dagster daemon and a real `send_message` function, so it isn't executed in the book.

46.7 Delivery only after the data passes its checks

This is the most important design rule in this chapter: **nothing leaves the pipeline until the data has passed its checks**. Reports, emails, dashboards refreshes that people act on, and syncs into the CRM all come *after* the checks, and depend on them.

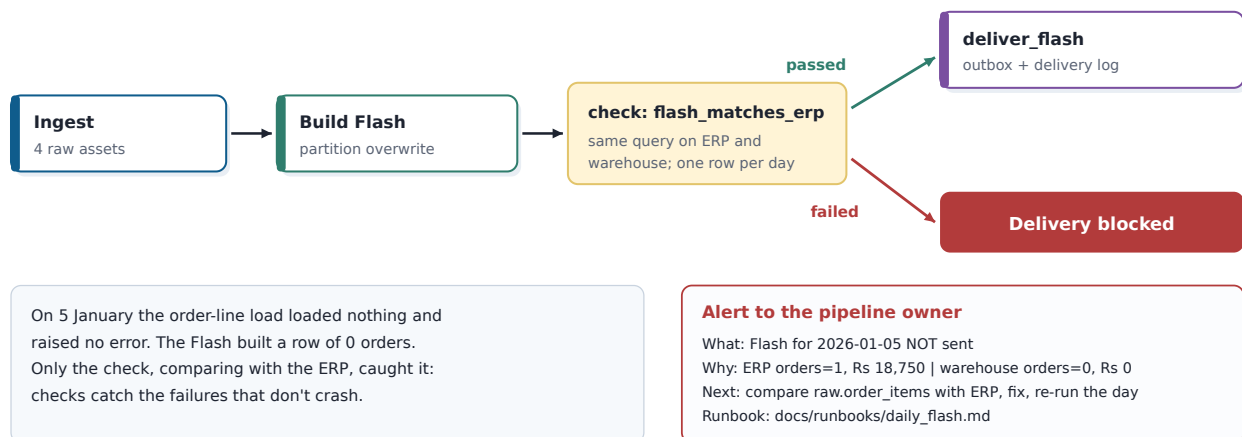


Figure 46.4 – Checks gate delivery. When a check fails, the report isn't sent, and the owner gets an alert that says what to do.

A bad morning

It's 5 January. The ERP has had its day (`apply_day(2)`), and two things go wrong at once: a bug in the order-line load means it loads nothing, and Bhiwandi Main has changed the columns in its dispatch file (Chapter 45, section 45.8).

```

apply_day(2)
ingest.BROKEN_ITEMS_SYNC = True          # simulate a faulty order-line load

result = dg.materialize(ALL, partition_key="2026-01-05", raise_on_error=False)
show_log()
print("run succeeded:", result.success)
print("files in outbox:", sorted(os.listdir("outbox")))

```

```

[2026-01-05] raw_orders: 1 new, 1 changed, 0 deleted
[2026-01-05] raw_order_items: 0 new, 0 changed, 0 deleted
[2026-01-05] raw_dispatch: FAILED, schema change in exports/dispatch_2026-01-05.csv: missing
['qty_units']
[2026-01-05] daily_flash: orders=0, revenue=Rs 0.00
[2026-01-05] check_flash_matches_erp: FAILED (ERP: orders=1, revenue=Rs 18,750.00 | warehouse:
orders=0, revenue=Rs 0.00)
run succeeded: False
files in outbox: ['flash_2026-01-02.txt', 'flash_2026-01-02_correction_1.txt']

```

Reading it, step by step.

- `raw_orders` loaded the new order 10178 and the change to order 10176, correctly.
- `raw_order_items` loaded **nothing**: the simulated bug. Nothing in its own output says it's wrong: "0 new" is a normal message on a quiet day.
- `raw_dispatch` **failed loudly** with a clear message: the expected `qty_units` column is missing. That's the column check from Chapter 45 doing its job.
- `raw_crm_leads` succeeded.
- `daily_flash` **ran**, because both of its dependencies succeeded, and built a row with 0 orders and ₹0.00, because the new order had no lines in the warehouse.
- The check **failed**: the ERP says 1 order for ₹18,750.00; the warehouse says 0.
- `deliver_flash` **didn't run**. The outbox still holds only the 2 January report. No manager received a Flash saying Riverstone sold nothing on 5 January.

Notice two things. First, the dispatch failure didn't stop the Flash, because the Flash doesn't depend on dispatches: the dependency graph limited the damage. Second, the order-line bug didn't raise any error at all. Only the check, comparing the result against the ERP, caught it. **Checks catch the failures that don't crash.**

Watch out: a blocked delivery isn't a clean table. The check stopped the email, but `mart.daily_flash` now holds a wrong row for 5 January, and anyone querying that table directly would see it. Chapter 47 covers the **write-audit-publish** pattern, which builds new data in a staging area, checks it, and only then makes it visible, so failed data never reaches readers at all.

An alert someone can act on

```
def alert_on_failure(result, day):
    failed_checks = [c.check_name for c in result.get_asset_check_evaluations() if not c.passed]
    failed_steps = sorted({e.step_key for e in result.all_events if e.event_type_value ==
"STEP_FAILURE"})
    if result.success:
        return None
    message = (f"Daily Sales Flash for {day} was NOT sent.\n"
              f"Failed checks: {failed_checks or 'none'}\n"
              f"Failed steps: {failed_steps}\n"
              f"Next step: compare raw.order_items with the ERP, fix, then re-run partition
{day}.\n"
              f"Runbook: docs/runbooks/daily_flash.md")
    with open(f>alerts/flash_{day}.txt", "w") as f:
        f.write(message)
    return message

print(alert_on_failure(result, "2026-01-05"))
```

```
Daily Sales Flash for 2026-01-05 was NOT sent.
Failed checks: ['flash_matches_erp']
Failed steps: ['daily_flash_flash_matches_erp', 'raw_crm_leads', 'raw_dispatch']
Next step: compare raw.order_items with the ERP, fix, then re-run partition 2026-01-05.
Runbook: docs/runbooks/daily_flash.md
```

How it works. `alert_on_failure()` reads the run's result: which checks failed, and which steps failed. It writes a message that says what didn't happen (the Flash wasn't sent), why, what to do next, and where the runbook is. In production, the same message would be sent by email or chat, as in the sensor example in section 46.6.

Note the step names: Dagster names a check step after its asset and check, `daily_flash_flash_matches_erp`.

Fixing and re-running

The pipeline owner fixes the order-line bug, and confirms with Bhiwandi Main that the renamed column `units` means the same as `qty_units`. Both fixes are small; the re-run is one command:

```
ingest.BROKEN_ITEMS_SYNC = False
ingest.CONFIRMED_RENAMES = {"units": "qty_units"} # confirmed with Bhiwandi Main
result = dg.materialize(ALL, partition_key="2026-01-05", raise_on_error=False)
show_log()
print("run succeeded:", result.success)
```

```
[2026-01-05] raw_orders: 0 new, 0 changed, 0 deleted
[2026-01-05] raw_order_items: 1 new, 0 changed, 0 deleted
[2026-01-05] raw_dispatch: 2 rows
[2026-01-05] daily_flash: orders=1, revenue=Rs 18,750.00
[2026-01-05] check flash_matches_erp: PASSED (ERP: orders=1, revenue=Rs 18,750.00 | warehouse:
orders=1, revenue=Rs 18,750.00)
[2026-01-05] deliver_flash: wrote outbox/flash_2026-01-05.txt
run succeeded: False
```

Reading it. The order-line load picked up the missing line for order 10178. The dispatch file loaded with the confirmed column mapping. The Flash was rebuilt at 1 order for ₹18,750.00, the check passed, and the report was delivered. Because every step is idempotent, re-running the whole day was safe: `raw_orders` found nothing new to do.

What to check before delivery

Check	Catches	Chapter
Reconciliation with the source (counts, totals)	Loads that silently miss or duplicate data	45, this section
Uniqueness of keys (one row per day, per order)	Double-counting from non-idempotent steps	46.4, 47
Freshness: the newest data is from the expected day	Stale sources, stuck loads	47
Volume: row counts within a normal range	Empty loads that don't error	47
Validity: no negative quantities, known statuses only	Bad data from sources	47
Schema: expected columns present	Source changes	45

46.8 The same pipeline in Airflow

Apache Airflow is the orchestrator most often named in data engineering job descriptions. It's task-based: you define a DAG of tasks, and each task runs a function or a command. Here's Riverstone's pipeline written for Airflow 2, using its TaskFlow style.

```

# dags/daily_flash.py (Airflow 2.x, TaskFlow API)
import pendulum
from datetime import timedelta
from airflow.decorators import dag, task
from airflow.exceptions import AirflowFailException

import ingest, flash                                # the same companion functions

@dag(
    schedule="30 6 * * *",
    start_date=pendulum.datetime(2026, 1, 1, tz="Asia/Kolkata"),
    catchup=False,                                   # don't run every missed day automatically
    default_args={"retries": 3, "retry_delay": timedelta(seconds=60)},
    tags=["riverstone", "daily-flash"],
)
def daily_flash_pipeline():

    @task
    def raw_orders(ds=None):                          # ds = the run's logical date, "YYYY-MM-DD"
        return ingest.sync_table(flash.warehouse(), "orders")

    @task
    def raw_order_items(ds=None):
        return ingest.sync_table(flash.warehouse(), "order_items")

    @task
    def build_flash(ds=None):
        flash.build_flash_safely(ds)                  # delete-then-insert in one transaction

    @task
    def check_flash(ds=None):
        if not flash.matches_erp(ds):
            raise AirflowFailException(f"Flash for {ds} does not match the ERP") # fail, no retry

    @task
    def deliver_flash(ds=None):
        flash.deliver(ds)                             # skips if already delivered

    built = build_flash()
    [raw_orders(), raw_order_items()] >> built >> check_flash() >> deliver_flash()

daily_flash_pipeline()

```

How it maps to the Dagster version.

Idea	Dagster (this chapter)	Airflow
Unit of work	Asset (a table or file that should exist)	Task (a function to run)
Dependencies	<code>deps=[...]</code> on the asset	<code>>></code> between tasks
Which day	<code>context.partition_key</code>	<code>ds</code> (the logical date), or <code>data_interval_start</code>
Schedule	<code>build_schedule_from_partitioned_job</code>	<code>schedule=</code> on the DAG
Time zone	On the partitions definition	On <code>start_date</code> (a time-zone-aware date)
Retries	<code>RetryPolicy</code> per asset	<code>retries</code> and <code>retry_delay</code> in <code>default_args</code> or per task
Checks that block delivery	<code>@asset_check(blocking=True)</code>	A check task upstream of delivery; <code>AirflowFailException</code> fails without retrying

Idea	Dagster (this chapter)	Airflow
Backfill	Select partitions; dagster job backfill	<code>airflow dags backfill -s 2026-01-01 -e 2026-01-05 daily_flash_pipeline</code>
Run one day for testing	<code>materialize(..., partition_key=...)</code>	<code>airflow dags test daily_flash_pipeline 2026-01-02</code>

Watch out: Airflow's logical date is the start of the interval. For a daily DAG, the run that happens on the morning of 3 January has a logical date (`ds`) of 2 January, because it processes the interval that *started* on 2 January. This surprises almost everyone the first time. Test with `airflow dags test` and print `ds` before trusting it.

Simplification note. This Airflow code isn't executed in the book: Airflow needs its own installation, a metadata database, and a scheduler process, and it imports helper modules (`flash`) that you would write from this chapter's Dagster code. The structure and settings follow Airflow 2's documented TaskFlow API; check the documentation for your version, as Airflow 3 changed several details.

Interview extra point. If an interviewer asks about Airflow and you've used Dagster (or the reverse), map the ideas: DAG and dependencies, logical date or partition, retries, backfills with side effects excluded, checks before delivery, and idempotent tasks. Interviewers care far more about whether you'd design the pipeline safely than about which decorator you'd type. Chapter 77 includes pipeline design questions.

46.9 Choosing an orchestrator

Option	Model	Strengths	Watch for	A good fit when
Cron / Task Scheduler	Time-based commands	Simple, everywhere, nothing to run	No dependencies, retries, history, or backfills; failures are silent unless you add alerts	One or two independent scripts with an owner who checks them
Apache Airflow	Task DAGs	Very widely used; huge library of operators; managed versions on every major cloud	Heavier to run; the logical-date model; asset lineage is less central	Teams with many varied workflows, or where the platform already runs it
Dagster	Asset graphs	Assets, partitions, and checks built in; quick local testing; clear lineage	Smaller community than Airflow; concepts to learn	Data platforms built around tables and models, including dbt projects
Prefect	Python flows	Pythonic and lightweight; good for dynamic workflows	Fewer data-asset features built in	Python-heavy teams wanting minimal ceremony

Option	Model	Strengths	Watch for	A good fit when
Cloud schedulers and workflow services	Varies	Managed, integrated with one cloud	Tied to one provider; features vary	Small pipelines already on one cloud (Chapter 52)
Built-in schedulers in tools (dbt Cloud, warehouse tasks, BI refresh schedules)	Per tool	Nothing extra to run	Each tool schedules only itself; cross-tool dependencies get lost	A single tool's work, not a cross-system pipeline

A few questions settle most choices:

1. **How many steps depend on each other, across how many systems?** Few and independent: a scheduler may be enough. Many and connected: an orchestrator.
2. **What does the team already run and know?** A well-run Airflow beats a half-adopted new tool.
3. **Is the work about data assets (tables, models) or about actions (jobs, scripts)?** Assets lean toward Dagster; varied actions lean toward Airflow or Prefect.
4. **Who will operate it?** Self-hosting any orchestrator means upgrades, a metadata database, and monitoring. Managed services cost money and save that effort.

Real-life example: the report that outgrew cron. Many teams start exactly like Riverstone: a handful of cron jobs at 6:00, 6:15, and 6:30, spaced to "leave enough time". It works until month-end volumes make the 6:00 job run long, and the 6:30 report quietly uses half-loaded data. Moving to an orchestrator usually starts with that one incident, and the first win is replacing time gaps with real dependencies.

46.10 Operating pipelines

A pipeline that runs is not the same as a pipeline that's operated well. A few habits make the difference.

- **Every pipeline has an owner.** A named person or team who gets the alerts and decides what happens when it fails.
- **Every important output has a deadline.** A **service-level agreement** (SLA) or objective states when data must be ready, such as "Daily Sales Flash delivered by 7:00 a.m. IST on working days". Alert when the deadline is at risk, not only when a step fails.
- **Every known failure has a runbook.** What the alert means, how to check the cause, how to fix it, how to re-run safely, and whom to tell. The alert in section 46.7 links to one.
- **Every re-run and backfill is recorded.** Who ran what, for which dates, and why.
- **Changes are tested before they're deployed.** Run the pipeline for a known partition in a test environment and compare outputs with the previous version (Chapter 29's testing habits, applied to data).
- **Pipelines are code.** Definitions live in Git, go through review, and are deployed the same way every time (Chapter 52).

- **Downstream readers know the status.** When the Flash is late, tell the sales team before 7:00, rather than letting them discover it.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Spacing cron jobs by time instead of dependencies	Reports built on half-loaded data on busy days	Declare dependencies in an orchestrator (46.1)
Leaving schedules in the server's time zone	Runs at the wrong local time; "yesterday" is wrong for hours each night	Set Asia/Kolkata (or your zone) explicitly (46.2)
Steps that read the clock instead of the partition	Backfilled days get today's numbers	Pass the partition date into every step
Appending results in a step that can be retried	Doubled rows and revenue after a failure (46.4)	Partition overwrite in a transaction, or upserts
Assuming idempotency without testing it	Surprise duplicates at the first real failure	Crash a step on purpose and re-run (46.4)
Sending emails or syncs before checks	Wrong numbers reach managers or the CRM	Delivery last, blocked by checks (46.7)
Re-delivering history during backfills	Dozens of old reports arrive at once	Exclude side effects from backfills (46.5)
Silently overwriting delivered numbers after a correction	People act on the old figure without knowing it changed	Send a labeled correction; keep a delivery log (46.5)
Retrying every failure	Bugs hidden by lucky retries; wrong passwords retried until accounts lock	Retry transient errors only; log and review retries (46.6)
No timeouts	A stuck run blocks everything and no alert fires	Timeouts on calls, statements, and runs
Alerts without context	People ignore alerts or can't act on them	What failed, impact, next step, runbook link (46.7)
Trusting "0 rows loaded"	Empty loads pass unnoticed	Reconciliation and volume checks (46.7, Ch 47)
Making everything depend on everything	One broken file stops every report	Declare only real dependencies (Figure 46.1)
Pipelines without owners or deadlines	Failures noticed by the business, days later	Owner, SLA, runbook (46.10)

In the real world: the Flash that went out twice

Two months after the pipeline went live, Anita forwarded Meera an email at 7:10 a.m. with one line: "Which one is right?"

Below it were two copies of the Daily Sales Flash for the previous day, sent eleven minutes apart. The first said ₹3,12,400 in bookings. The second said ₹3,24,900.

Meera opened the run history. The 6:30 run had failed at 6:41 when the warehouse connection dropped, and the orchestrator had retried it automatically. Both runs had reached delivery. The difference was easy to explain once she saw it: between the first and second runs, a sales executive had entered a late order from the previous afternoon, so the second run's numbers were more complete. But the email step had no memory. It sent whatever it was asked to send, every time it ran.

"So the second one is right," Anita said when Meera called her. "But the managers don't know that. Two of them have already replied to the first one."

Meera and the data engineer made three changes that week.

First, **delivery got a memory**. Every sent report was recorded with a hash of its content. An identical report wouldn't be sent twice. A report whose numbers changed would be sent with the subject line "CORRECTION: Daily Sales Flash", and a first line saying what changed and why.

Second, **delivery moved behind a cut-off check**. The Flash now waited until the ERP's end-of-day close flag was set before building, so late entries from the previous afternoon were included in the first send, not discovered by a retry.

Third, **retries became visible**. Any run that needed a retry now posted a one-line note in the data team's channel, so a flaky warehouse connection would be noticed and fixed, not silently absorbed.

The next month, the warehouse connection dropped again during a run. The retry succeeded, the content hash matched what had already been prepared, and nothing was sent twice. The only sign was a single line in the data team's channel, which led the engineer to a network setting that had been causing the drops all along.

What made this work.

- **Meera read the run history before guessing.** The orchestrator's record of both runs explained the duplicate in minutes.
- **They fixed the design, not only the incident:** idempotent delivery, a clear correction path, and a real readiness condition instead of a clock time.
- **They made retries visible**, so the underlying problem got fixed instead of hidden.

Tools

- **Dagster** (`pip install dagster`); `dagster dev -f yourfile.py` opens the local web interface. For deployed use, Dagster also needs its daemon for schedules and sensors.
- **Python 3.12**, **DuckDB**, **psycopg2**, **requests**, and **PostgreSQL 16**, as in Chapter 45.
- **The Chapter 46 companion folder** (`companion/ch46/`): `reset_ch46.py`, `ingest.py`, and the three helpers copied from Chapter 45. Run Python from that folder. Set `RIVERSTONE_SOURCE` if your PostgreSQL needs a user or password.
- **Versions used for the outputs shown:** Python 3.12.3, Dagster 1.13.23, DuckDB 1.5.5, psycopg2 2.9.13, requests 2.33.1, PostgreSQL 16.

- **Apache Airflow** (not needed for this chapter): the official quick-start runs it locally with a single command; managed versions exist on the major clouds.
-

The project: orchestrate Riverstone's ingestion

Goal: turn your Chapter 45 ingestion job into an orchestrated, partitioned, tested pipeline that delivers the Daily Sales Flash only when the data is right. Chapter 47 will add a fuller set of quality checks and monitoring on top.

Option A: Riverstone. Use the companion environment and your Chapter 45 project.

Option B: your own pipeline. Use a report you currently produce on a schedule, with data you have permission to use. Keep credentials out of code.

Steps

1. **Draw the graph first.** List every asset (raw tables, staging tables, the Flash, delivery) and its real dependencies. Mark which failures should stop which outputs.
2. **Define daily partitions** in `Asia/Kolkata` (or your time zone), and make every asset use the partition date, never the clock.
3. **Wrap your Chapter 45 loads as assets.** Add `customers` and `products` as full loads.
4. **Build the Flash** with partition overwrite in a transaction. Add a second measure of your choice, such as orders by segment.
5. **Add at least two blocking checks:** reconciliation with the ERP, and exactly one row per day.
6. **Add delivery** with a delivery log and a correction path.
7. **Add a retry policy** to the API step only, and log every retry.
8. **Write the failure test:** crash the Flash step after writing, re-run, and assert that there's one row and the revenue matches the ERP. Then do the same with an append version and record what goes wrong.
9. **Run a backfill** for 1–5 January without delivery, and show the outbox is unchanged.
10. **Simulate a bad morning:** break one load, confirm the check blocks delivery, and produce an alert with a next step and a runbook link. Write the runbook (half a page).
11. **Schedule it** for 6:30 IST, open `dagster dev`, and take a screenshot of the graph and a successful run.

Stretch goals

- Translate your pipeline to Airflow and run it with `airflow dags test` for 2 January.
 - Add a 7-day look-back: each daily run also rebuilds the previous six days, and only sends corrections when numbers change.
 - Add a deadline: if the Flash hasn't been delivered by 7:00 IST, write an alert even if nothing has failed.
-

You've got it when...

- ☐ You can explain what an orchestrator adds to cron, with at least four examples.
 - ☐ You can draw a pipeline as a DAG and explain why only real dependencies belong in it.
 - ☐ You can explain the difference between task-based and asset-based orchestration.
 - ☐ You can read and write cron expressions, and set time zones explicitly.
 - ☐ You can build partitioned assets in Dagster that use the partition date instead of the clock.
 - ☐ You can prove a step is idempotent with a deliberate failure test, and explain why the append version doubled revenue.
 - ☐ You can list idempotency patterns for tables and for side effects such as emails.
 - ☐ You can run a backfill that rebuilds history without re-delivering it.
 - ☐ You can handle a late correction by re-running one partition and sending a labeled correction.
 - ☐ You can explain when retries help, when they hide bugs, and why timeouts matter.
 - ☐ You can make checks block delivery, and explain why the failed order-line load raised no error.
 - ☐ You can write an actionable alert with a next step and a runbook link.
 - ☐ You can map the pipeline between Dagster and Airflow, including Airflow's logical date.
 - ☐ You can recommend an orchestrator for a given team and explain why.
-

Recap

- An **orchestrator** runs pipeline steps in dependency order, on schedule, with retries, history, backfills, and alerts. **Cron** is enough for simple, independent jobs.
 - A pipeline is a **DAG**. **Task-based** tools (Airflow) describe what to run; **asset-based** tools (Dagster) describe what data should exist.
 - **Schedules** use **cron expressions** and need explicit **time zones**. **Partitions** split runs by day, so each run knows exactly what it's building.
 - **Idempotency** must be proven with a **failure test**. An append step retried after a crash doubled revenue to ₹77,420.00; **partition overwrite in a transaction** kept it at ₹38,710.00.
 - **Side effects** (emails, syncs) can't be rolled back: put them last, behind checks, with a **delivery log**.
 - **Backfills** rebuild history; exclude delivery. **Late corrections** re-run one partition and send a labeled correction.
 - **Retries** handle transient failures, and can hide bugs; log them. **Timeouts** stop runs that never end.
 - **Checks that block delivery** catch failures that don't crash: an order-line load that loaded nothing raised no error, and only the reconciliation check stopped a ₹0 Flash reaching managers.
 - Good **alerts** are actionable, rare, routed, and linked to a **runbook**.
 - Airflow expresses the same design with tasks, `ds`, retries, and a check task before delivery.
 - Operate pipelines with **owners**, **SLAs**, **runbooks**, recorded re-runs, and code review.
-

Practice exercises

Run `reset()` and re-run the definitions blocks before each exercise that uses the companion environment, unless it says otherwise.

Warm-up

1. Write cron expressions for: (a) 7:15 every weekday (Monday to Friday); (b) every 30 minutes; (c) 9:00 on the first day of every quarter (January, April, July, October).
2. A schedule is set to `0 1 * * *` in UTC. At what time does it run in Mumbai (IST)? Which calendar date is it in IST when it runs?
3. For each step, say whether it's idempotent as described, and why: (a) `INSERT INTO mart.daily_flash ...` for the partition day; (b) `INSERT OR REPLACE` by primary key; (c) sending an email every time the step runs; (d) delete the partition's rows, then insert, in one transaction.

Core

4. In section 46.4, the naive table ended with two rows for 2 January. If the naive step had been retried three times after three crashes, and then succeeded, how many rows would there be, and what revenue would a `SUM` report?
5. Add a second blocking check to the pipeline, `one_flash_row_per_day`, that passes only when `mart.daily_flash` has exactly one row for the partition. Why is this check useful even though `flash_matches_erp` already compares the row with the ERP?
6. In section 46.7's bad morning, why did `daily_flash` run at all, and why did `raw_dispatch`'s failure not stop it? Draw (or describe) the dependency change that would make a dispatch failure block the Flash, and argue whether that would be a good idea.
7. After the correction in section 46.5, what would `deliver_flash` do if the ERP were corrected *back* to 6 crates and the 2 January partition re-run? Trace it through the delivery log.
8. Rewrite the alert in section 46.7 for a different failure: the CRM API returned 401 Unauthorized on every attempt. Include what failed, the impact on the Flash, the next step, and a runbook link.
9. A colleague proposes adding `retry_policy=dg.RetryPolicy(max_retries=5)` to `daily_flash` "to make the pipeline more reliable". Explain what this would and wouldn't help with, using the section 46.7 bad morning as an example.

Stretch

10. Design a 7-day look-back for the Flash: each morning's run rebuilds today's partition and the previous six, but only delivers today's Flash and any *corrections* to earlier days. Describe the assets and selections, and what the delivery log must store.
11. Riverstone's month-end finance pack must use numbers that don't change after the 5th of the following month. Describe how you'd combine partitions, a closing rule, and checks so late corrections after the 5th are reported for review rather than silently changing the pack.
12. Translate the section 46.7 check-then-deliver behavior into Airflow terms precisely: which task fails, which exception, which retries apply, and what state the delivery task ends in.

Think about it (no code needed)

- l3. Riverstone's data team has six cron jobs today and one part-time engineer. Make the case for staying with cron for another six months, then the case for moving to an orchestrator now. Which would you choose, and what single incident would change your mind?
 - l4. Why is "delivery last, behind checks" a rule for more than emails? Give two examples from Chapter 51's world (writing data back into business systems) where breaking it would be worse than a wrong email.
-

Key terms

orchestrator · pipeline · cron · directed acyclic graph (DAG) · task · asset · dependency · schedule · cron expression · time zone (UTC, IST) · partition · partition key · job · retry policy · asset check · blocking check · idempotent · failure test · transaction (BEGIN, COMMIT, ROLLBACK) · partition overwrite · delivery log · idempotency key · side effect · backfill · late-arriving data · look-back window · transient failure · timeout · alert · runbook · run failure sensor · logical date (Airflow `ds`) · catchup · service-level agreement (SLA) · write-audit-publish

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 47, Data Quality, Observability & Contracts**, turns this chapter's checks into a full testing and monitoring layer: freshness, volume, and validity checks, write-audit-publish, data contracts, and alert fatigue.
 - **Chapter 48, Big Data & Distributed Compute**, runs heavier steps on Spark and DuckDB at scale, inside pipelines like this one.
 - **Chapter 49, Storage, Warehouses & Lakehouses**, explains the transactions (ACID) and table formats that make partition overwrites safe at scale.
 - **Chapter 50, Streaming & Real-Time**, covers pipelines that run continuously instead of once a day.
 - **Chapter 51, Data Activation**, adds CRM and ERP syncs as downstream steps, gated by checks and made idempotent with delivery logs and idempotency keys.
 - **Chapter 52, The Cloud, Containers & Infrastructure as Code**, deploys pipelines like this one: containers, secrets, and environments.
 - **Chapter 63, Designing Automation & Integration Architecture**, looks at orchestration across a whole company: ownership, governance, and choosing tools.
 - **Part VIII**: pipeline and orchestration questions appear in the data engineering and system design interview chapters (Chapter 77).
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

- (a) `15 7 * * 1-5`. (b) `*/30 * * * *`. (c) `0 9 1 1,4,7,10 *`. The common slip in (a) is `* 7 * * 1-5`, which runs every minute from 7:00 to 7:59.
- 1:00 UTC is **6:30 a.m. IST** (UTC + 5:30), on the **same calendar date**. The reverse trap is a schedule at 20:00 UTC, which runs at 1:30 a.m. IST on the *next* day.
- (a) **Not idempotent**: each run inserts another row for the day, as section 46.4 showed. (b) **Idempotent**: the same keys end with the same values however often it runs. (c) **Not idempotent**: each run sends another email; the delivery log in section 46.3 is what makes it safe. (d) **Idempotent**: every run leaves exactly the rows for that partition, and a failure rolls back.
- Each of the three crashed runs inserted a row before crashing, and the successful run inserted one more: **4 rows**. A `SUM` would report $4 \times ₹38,710.00 = ₹154,840.00$.
- A sample check:

```
@dg.asset_check(asset=daily_flash, blocking=True)
def one_flash_row_per_day(context):
    day = context.partition_key
    n = wh.execute("SELECT COUNT(*) FROM mart.daily_flash WHERE flash_date = ?", [day]).fetchone()
    [0]
    return dg.AssetCheckResult(passed=(n == 1), metadata={"rows": n})
```

It's useful because it names the problem precisely. `flash_matches_erp` also fails when there are duplicate rows, but its message only says the numbers don't match; the uniqueness check says *why*, which points straight to a non-idempotent step. Separate checks for separate failure modes make alerts actionable. It also protects readers who query the table directly and sum it.

6. `daily_flash` depends only on `raw_orders` and `raw_order_items`, and both **succeeded** (the order-line load loaded nothing, but didn't raise an error). `raw_dispatch` isn't a dependency of the Flash, so its failure only affects assets downstream of it. To make a dispatch failure block the Flash, you'd add `raw_dispatch` to `daily_flash`'s `deps`. That would usually be a **bad idea**: the Flash reports bookings, which don't use dispatch data, so a warehouse spreadsheet problem would stop the sales team's numbers for no reason. Add a dependency only when the output really uses the input; a dispatch report, by contrast, *should* depend on `raw_dispatch`.

7. The Flash would be rebuilt at ₹38,710.00. `deliver_flash` computes the content hash and finds that **exactly this content was already delivered** (the original 2 January report, recorded on the first run), so it logs "already delivered, skipped" and sends nothing. The managers' latest message is still the correction for ₹40,110.00, which is now wrong. That's a gap in this simple design: "already delivered" should compare with the *most recent* delivery, not any past delivery. A fix is to compare the new hash with the latest entry for that day, and send a second correction when they differ.

8. A sample alert: "Daily Sales Flash for 2026-01-05: CRM leads were NOT loaded. The CRM API returned 401 Unauthorized on all 4 attempts (the token is missing, wrong, or expired). Impact: the Flash itself is unaffected and has been delivered; lead figures in the weekly pipeline report will be stale until this is fixed. Next step: check whether the CRM API token was rotated, update the secret `CRM_API_TOKEN`, then

re-run partition 2026-01-05 for `raw_crm_leads` only. Runbook: docs/runbooks/crm_ingestion.md." A good answer says the Flash wasn't affected, because `raw_crm_leads` isn't one of its dependencies, and notes that 401 shouldn't be retried at all.

9. Retries on `daily_flash` would help only if the step itself failed **transiently**, for example a dropped warehouse connection during the build, and only because the step is idempotent. They wouldn't help on the bad morning at all: `daily_flash` **didn't fail**. It succeeded with wrong numbers, because its input was incomplete. The problem was caught by the check, and retrying a check that compares wrong data with the ERP gives the same failure every time. Five retries would also delay the alert. Reliability comes from checks, idempotency, and fixing root causes; retries only cover transient faults.

10. One design: keep the daily partitions. The morning run materializes ingestion once (it isn't per-day), then `daily_flash` and its checks for **seven partition keys**: today and the previous six. Delivery runs for **today's** partition as normal, plus a `deliver_corrections` step that, for each of the six earlier days, compares the new content hash with the **latest** delivery for that day and sends a labeled correction only when it differs. The delivery log must store the day, the content hash, the time sent, and the type (original or correction), so "latest" can be found. Checks must pass for every rebuilt day before any correction is sent.

11. One approach: keep daily (or monthly) partitions and a **closing table** that records when each month was closed (the 5th of the following month). The pipeline still re-runs late partitions when source data changes, but a **blocking check** on the month-end pack compares the new numbers with the closed version: if the month is closed and numbers changed, the check fails *for the pack*, the pack isn't regenerated, and an alert goes to finance with the difference and the orders involved. Finance then decides whether to reopen the month or book an adjustment in the next one. The daily Flash can still show corrected operational numbers; the closed pack stays as reported.

12. In Airflow, the check is a task (`check_flash`) upstream of `deliver_flash` . When the numbers don't match, it raises `AirflowFailException` , which marks the task **failed immediately, without using its retries**. (A normal exception would be retried according to `retries` , which is wasteful for a data mismatch.) Because `deliver_flash` depends on `check_flash` with Airflow's default trigger rule (`all_success`), it ends in the **upstream_failed** state and never runs. The DAG run is marked failed, and a failure callback or alerting integration notifies the owner.

13. *Stay with cron*: six jobs and one part-time engineer means an orchestrator adds a system to install, upgrade, and learn; if the jobs are independent and each has alerts and a runbook, the risk is low, and the engineer's time may be better spent on checks. *Move now*: the Flash already depends on several loads, cron spacing will fail on a busy day, and adding checks, retries, and backfills by hand in six scripts costs more than adopting a tool once; a managed or simple local deployment keeps the operational burden small. A reasonable choice is to move the Flash's steps to an orchestrator now and leave truly independent jobs on cron. The incident that would change a "stay" decision: any day when a report goes out built on half-loaded or stale data because a job ran long.

14. Because anything that leaves the pipeline changes something outside it, and many of those changes can't be undone. Examples: (1) syncing **credit limits** from the warehouse into the ERP: a failed load that sets limits to zero would block real customers' orders until someone notices; (2) pushing **lead scores** or "customer at risk" flags into the CRM: wrong scores would send sales reps to the wrong customers and trigger automated emails, and the CRM keeps the history. Other good examples

include reorder flags that trigger purchase orders, and marketing lists that send messages. In each case, a check that blocks the sync costs minutes; a wrong sync can cost orders, money, or customer trust.